AMS Operational Pipelines (v3 fragment)

54 behaviors, 54 relationships, 7 top-level pipelines

architecture-model-standard

2026-09-20

Overview

Seven top-level operational pipelines defining AMS as a running system. Source: `.architecture/model/pipelines.v3.yaml`. Flow annotations: `.architecture/model/pipeline-flow-annotations.yaml`. Generated by `scripts/print_pipelines_pdf.py`.

Top	Name	Direct subs	Realizes
BEH-P1	`run\_extraction\_pipeline`	11	CAP-PB
BEH-26	`deliver\_slice\_scoped\_context`	6	CAP-SL
BEH-27	`learn\_across\_pipeline\_runs`	6	CAP-DF
BEH-28	`run\_model\_driven\_development\_loop`	4	CAP-DF
BEH-LLM	`route\_llm\_calls\_via\_policy`	8	CAP-DF
BEH-SIL	`emit\_component\_scoped\_telemetry`	8	CAP-DF
BEH-FBK	`append\_and\_read\_feedback\_journals`	4	CAP-DF

Coverage: 54 behaviors, 84 promises, 69 verified (82%).

Legend

Flow diagrams overlay three marker kinds at each step. **Solid** markers (`L`, `LLM`, `SI&L`) indicate sites present in code today. **Starred** markers (`L*`, `LLM*`, `SI&L*`) are proposed / design intent — the aggregation view lets one diagram show reality overlaid with where the model should wire the next integration.

Marker	Meaning
`L`	Logging site (stdlib logging, journal writes, non-SI&L)
`LLM`	LLM provider call site
`SI&L`	SI&L instrumented site (`@sil_instrument` or `SILStore.append`)
$*$ suffix	Proposed / design intent, not yet in code
·	No marker at this step

Capability glossary (CAP)

ID	Description
`CAP-DF`	Evolve architecture via feedback + self-improvement
`CAP-PB`	Extract architecture from code (pipeline)
`CAP-SL`	Deliver slice-scoped context within token budget

Component glossary (COMP)

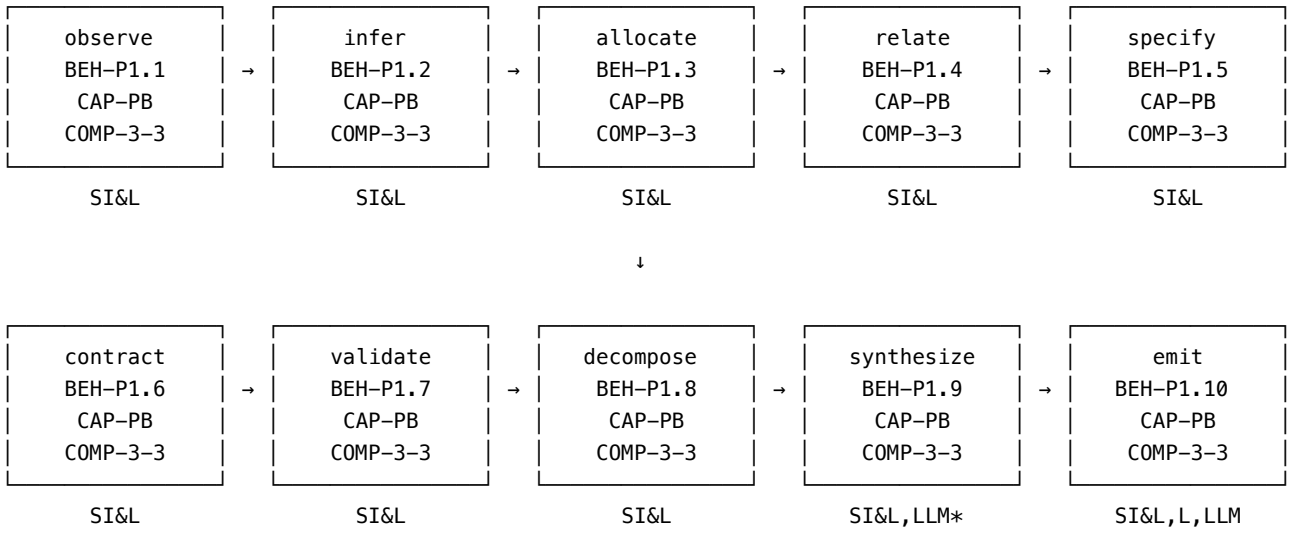
ID	Description
COMP-3-11	Architecture Model (cli)
COMP-3-12	Architecture Model (feedback)
COMP-3-14	Architecture Model (ai)
COMP-3-16	Architecture Model (orchestration)
COMP-3-18	Architecture Model (sil)
COMP-3-2	Architecture Model (comments)
COMP-3-3	Architecture Model (pipeline)
COMP-3-4	Architecture Model (llm)
COMP-3-5	Architecture Model (core)

BEH-P1 — run_extraction_pipeline

Orchestrate the 10-stage extraction pipeline (observe → infer → allocate → relate → specify → contract → validate → decompose → synthesize → emit) to produce .architecture-model.yaml from any Python source tree with zero manual configuration.

Realizes: CAP-PB

Flow



loopback: BEH-P1.9 → BEH-P1.1 (BEH-P1.R (recursive scoped sub-pipeline))

Promises

- **intent** *(verified)* — Orchestrate the 10-stage extraction pipeline (observe → infer → allocate → relate → specify → contract → validate → decompose → synthesize → emit) to produce .architecture-model.yaml from any Python source tree with z…
- **goals** *(verified)* — Deliver a validated architecture model from source code without hand-editing configuration; Preserve stage isolation so each stage is independently testable and resumable; Emit deterministic bytes given identical inpu…
- **requirements** *(verified)* — Stages execute in strict linear dependency order; Intermediate state persists to .architecture-models/cache/ between stages; Every stage records a report entry in pipeline-report.md; Emit stamps meta.provider when a p…
- **success_criteria** *(verified)* — Validator score configured threshold (default 80/100); Zero broken references in emitted model; All 10 stages record completion in pipeline-report.md; Component file coverage 90%
- **failure_modes** *(open)* — Stage cache corruption forces re-run of upstream stages; Circular imports break relate stage's dependency graph; Sub-system decomposition explodes on flat repos (mitigated by decompose.py threshold); Provider timeout …
- **constraints** *(verified)* — Must respect AMS_DETERMINISTIC_NOW for reproducibility guarantees; Every LLM call routed through the provider layer (BEH-LLM) — no direct HTTPS from stages; Every architecturally-significant stage entry point instrume…

Direct sub-behaviors (11)

ID	Name	Intent
BEH-P1.1	run_observe_stage	AST-scan source files to produce a module inventory with signatures, imports, routes, and constraint markers.

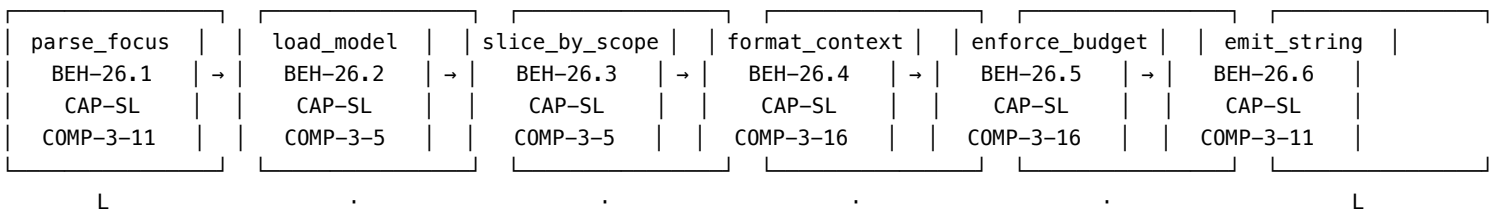
ID	Name	Intent
BEH-P1.2	run_infer_stage	Infer semantic entities (actors, capabilities, candidate behaviors) from raw observations using pattern rules and heuristics.
BEH-P1.3	run_allocate_stage	Allocate observed modules to components using multi-signal affinity (imports, name-prefix, directory).
BEH-P1.4	run_relate_stage	Derive typed relationships (realizes, depends-on, contains, exposes, constrained-by) between entities from observation and infe…
BEH-P1.5	run_specify_stage	Specify interface contracts by promoting routes to REST specs, argparse to CLI specs, and public APIs to library specs.
BEH-P1.6	run_contract_stage	Bind test contracts to components by mapping test files to their targets and extracting per-test expectations.
BEH-P1.7	run_validate_stage	Validate the intermediate model for structural integrity (ID uniqueness, referential integrity, orphan detection) and score 0–100.
BEH-P1.8	run_decompose_stage	Detect system boundaries by identifying components with 5 files that qualify as autonomous subsystems, splitting the emit targ…
BEH-P1.9	run_synthesize_stage	Run scoped sub-pipelines for each detected system to produce per-subsystem models with local coherence.
BEH-P1.10	run_emit_stage	Write all artifacts to disk (.architecture-model.yaml plus .architecture-models/<subsystem>/ tree) with deterministic bytes giv…
BEH-P1.R	run_recursive_subsystem_pipeline	Recursively invoke the extraction pipeline on each detected subsystem so nested systems get their own self-contained model plus…

BEH-26 — deliver_slice_scoped_context

Deliver a focused architectural context (single F-block, layer, or entity) to an LLM caller within a strict token budget, so the caller receives compressed evidence rather than the full model.

Realizes: CAP-SL

Flow



Promises

- **intent** *(verified)* — Deliver a focused architectural context (single F-block, layer, or entity) to an LLM caller within a strict token budget, so the caller receives compressed evidence rather than the full model.
- **goals** *(verified)* — Reduce prompt tokens 5× versus full-model delivery on typical queries; Preserve fragment digest stability across identical inputs
- **requirements** *(verified)* — Respect max_tokens caller budget without silent truncation; No dangling references after closure computation; Renderer emits deterministic bytes for identical scope+detail inputs
- **success_criteria** *(verified)* — Fragment digest stable across two identical slice calls; Zero broken cross-references in emitted fragment; Renderer output byte-identical across runs
- **failure_modes** *(open)* — Token budget too small to fit essential entities → hard-fail rather than silently drop; Scope references unknown entity id → KeyError with clear message; Overlay ordering perturbation breaks digest stability

Direct sub-behaviors (6)

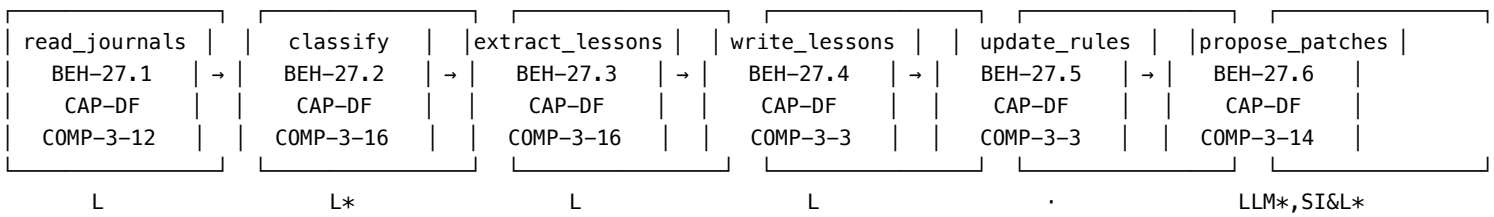
ID	Name	Intent
BEH-26.1	parse_focus_arg	Resolve the focus argument (all, F-block id, layer name, artifact name) into a typed scope descriptor for the slicer.
BEH-26.2	load_model	Load .architecture-model.yaml and merge v3 fragments into a queryable in-memory model.
BEH-26.3	slice_by_scope	Reduce the model to the requested scope via slice_by_fblock / slice_by_layer / slice_by_entity.
BEH-26.4	format_context	Render the sliced model as a deterministic context string using llm_context.format_model_context.
BEH-26.5	enforce_token_budget	Truncate or prioritize the formatted context to fit max_tokens, preserving highest-signal entities first.
BEH-26.6	emit_context_string	Return the budget-enforced context string to the caller (CLI/MCP/agent).

BEH-27 — learn_across_pipeline_runs

Accumulate lessons (heuristics, archetypes, workflows) from prior pipeline runs and apply them to subsequent extractions so quality improves monotonically across runs on the same repository.

Realizes: CAP-DF

Flow



Promises

- **intent** *(verified)* — Accumulate lessons (heuristics, archetypes, workflows) from prior pipeline runs and apply them to subsequent extractions so quality improves monotonically across runs on the same repository.
- **goals** *(open)* — Reduce repeat mistakes across runs by encoding pattern-fix pairs; Improve regen readiness grade run over run on the same repository; Keep learning-curve trend monotonically non-decreasing on happy-path repos
- **requirements** *(verified)* — Persist lessons to .architecture/learning/history.json; Deduplicate lessons by content-hash id; Apply lessons at stage entry (infer, allocate, relate) not at emit
- **success_criteria** *(open)* — Pattern hit-rate trending upward across successive runs; Regen readiness grade improving or stable; Zero oscillation (regression → improvement → regression) in learning curve

Direct sub-behaviors (6)

ID	Name	Intent
BEH-27.1	read_feedback_journals	Read append-only feedback journals (gates.jsonl, drift.jsonl, test_results.jsonl, llm_calls.jsonl) as ordered event streams.
BEH-27.2	classify_patterns	Classify observed events into pattern types (CROSS_DEP, MISSING_IMPL, WRONG_CONSTANT, API_MISMATCH, COMPLEX_BEHAVIOR, TEST_INFR…
BEH-27.3	extract_lessons	Derive deduplicated lessons from classified patterns, using content-hashed IDs so identical insights collapse.
BEH-27.4	write_lessons_md	Append new lessons to .architecture-models/lessons.md preserving append-only history.
BEH-27.5	update_auto_correct_rules	Feed accumulated lessons into pipeline auto-correction rules that adjust future runs.

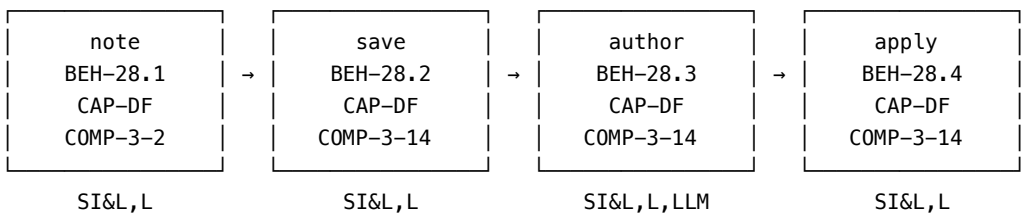
ID	Name	Intent
BEH-27.6	propose_model_patches	PROPOSED — use an LLM provider (R7-style) to author ModelPatch proposals that fill promise gaps flagged by the drift journal, f…

BEH-28 — run_model_driven_development_loop

Support the developer inner loop where a developer edits code, re-runs extraction, browses the v3 viewer, captures a promise-note, and files a change proposal — all without leaving the tool suite.

Realizes: CAP-DF

Flow



Promises

- **intent** *(verified)* — Support the developer inner loop where a developer edits code, re-runs extraction, browses the v3 viewer, captures a promise-note, and files a change proposal — all without leaving the tool suite.
- **goals** *(verified)* — Close the loop from a developer note to a validated model change; Zero context-switch cost between viewer, note, and proposal; Every change proposal is auditable back to the note that seeded it
- **requirements** *(verified)* — Notes persist under .architecture/comments/ with deterministic ids; Save auto-advances Job draft → approved → queued; Author walk is provider-agnostic (works with any LLMProvider Protocol impl)
- **success_criteria** *(open)* — Proposal applies without drift on model tip; Gate passes post-apply; SI&L failure_rate on affected components not regressing
- **failure_modes** *(open)* — Note targets a stale entity id (renamed between capture and save); Provider returns malformed ModelPatch → validator catches, job fails; Concurrent notes on the same entity attribute produce merge conflict

Direct sub-behaviors (4)

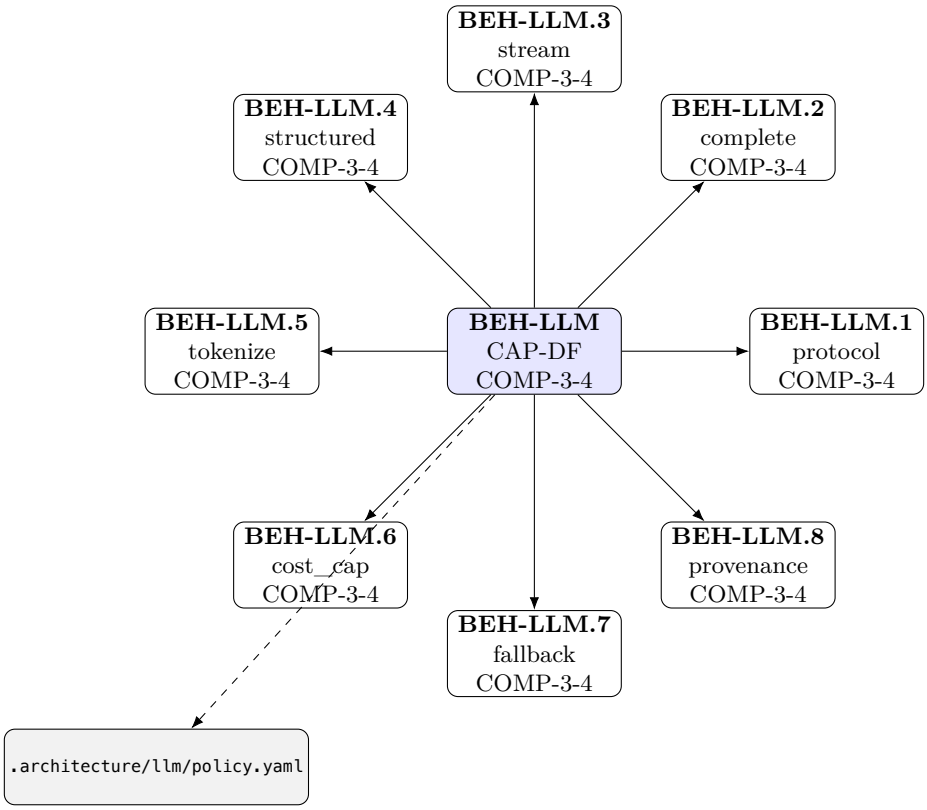
ID	Name	Intent
BEH-28.1	note_promise	Capture a free-text note attached to a promise (entity_id + attribute) via architect_promise_note; persists a CommentStub with …
BEH-28.2	save_promise_note	Convert a promise note into a WorkOrder + tracking Job (queued) via architect_promise_save; enqueue for authoring.
BEH-28.3	author_promise_content	Call an LLM provider to synthesize promise content into a Proposal via architect_promise_author; walks Job queued→running→valid…
BEH-28.4	apply_proposal	Apply a validated Proposal to the lifecycle store via architect_proposal_apply, publishing a new architecture generation.

BEH-LLM — route_llm_calls_via_policy

Route every LLM call through a provider-agnostic policy layer (.architecture/llm/policy.yaml) so callers name a TaskClass rather than a provider or model, enabling per-repo policy override without code changes.

Realizes: `CAP-DF`

Flow



Promises

- **intent** *(verified)* — Route every LLM call through a provider-agnostic policy layer (.architecture/llm/policy.yaml) so callers name a TaskClass rather than a provider or model, enabling per-repo policy override without code changes.
- **goals** *(open)* — Zero direct HTTPS calls from stages or walks — every LLM path routed through the provider layer; Cost cap enforcement per call and global budget cap; Fallback provider on transient error
- **requirements** *(verified)* — Providers implement the LLMProvider Protocol (complete, stream, structured, tokenize); Routing rules map TaskClass name → (provider, model, tier, max_cost_usd_per_call, fallback); Every call records a LLMCallRecord to…
- **success_criteria** *(verified)* — 100% of LLM calls in the codebase go through the policy layer (audited by CAP-WK.llm-endpoint-inventory); Zero calls exceed configured max_cost_usd_per_call; Fallback engages on 90% of transient provider errors

Direct sub-behaviors (8)

ID	Name	Intent
`BEH-LLM.1`	`expose\_provider\_protocol`	Expose the runtime-checkable LLMProvider Protocol (complete, stream, structured, tokenize) as the sole integration surface for …

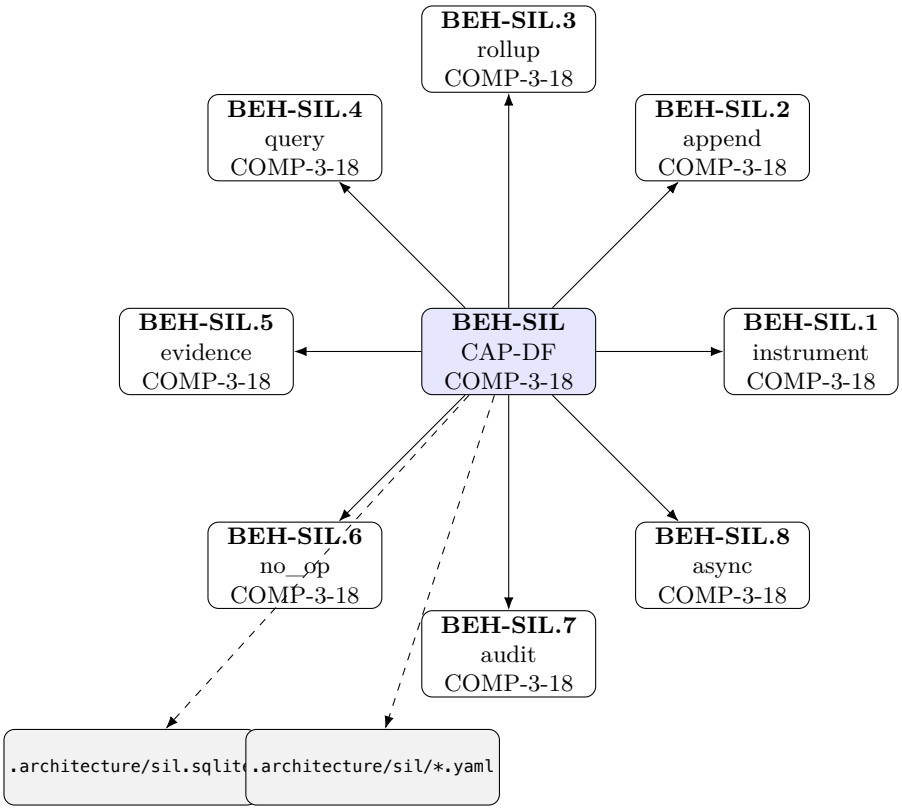
ID	Name	Intent
BEH-LLM.2	complete_prompt	Perform a single-shot completion via the resolved provider given a prompt and optional parameters.
BEH-LLM.3	stream_prompt	Stream tokens from the resolved provider so long-running calls surface progress and can be canceled early.
BEH-LLM.4	structured_output	Request structured JSON output validated against a caller-supplied schema so downstream consumers receive typed proposals.
BEH-LLM.5	tokenize_text	Return token count for arbitrary text so callers can enforce budget caps before dispatch.
BEH-LLM.6	enforce_cost_cap	Enforce per-call max_cost_usd_per_call and global_budget_usd budget caps before dispatch, blocking calls that would exceed either.
BEH-LLM.7	fallback_on_error	On provider error (timeout, 5xx, malformed response) retry via the fallback provider named in the routing rule, up to retry_bac…
BEH-LLM.8	record_call_provenance	Record every LLM call as an LLMCallRecord to .architecture/llm_calls.jsonl so the prompt-refinement walk (CAP-WK.prompt-refinem…

BEH-SIL — emit_component_scoped_telemetry

Emit component-scoped telemetry for architecturally-significant call sites (pipeline stages, renderers, validators, MCP tool handlers) so the model carries live evidence of runtime behavior.

Realizes: `CAP-DF`

Flow



Promises

- **intent** *(verified)* — Emit component-scoped telemetry for architecturally-significant call sites (pipeline stages, renderers, validators, MCP tool handlers) so the model carries live evidence of runtime behavior.
- **goals** *(verified)* — Turn the pipeline from an opaque batch process into an observable one; Provide 7-day trend evidence (invocations, failure_rate, avg_duration_ms) per component; Zero overhead when no SILStore is bound (decorator is no-op)
- **requirements** *(verified)* — Store events in a SQLite ring buffer at .architecture/sil.sqlite (50 most recent per component); Emit rollup snapshots to .architecture/sil/<component_id>.yaml with pre-aggregated 7-day metrics; Instrument every stage…
- **success_criteria** *(verified)* — Instrumentation coverage 100% on defined architectural surfaces (audited by tests/sil/test_decorator_coverage.py); Rollup snapshots regenerate deterministically on identical inputs; Ring buffer never grows beyond 50…

Direct sub-behaviors (8)

ID	Name	Intent
`BEH-SIL.1`	`instrument\_call\_site`	Wrap a call site with the @sil_instrument decorator so every invocation records a SILRecord to the bound store.

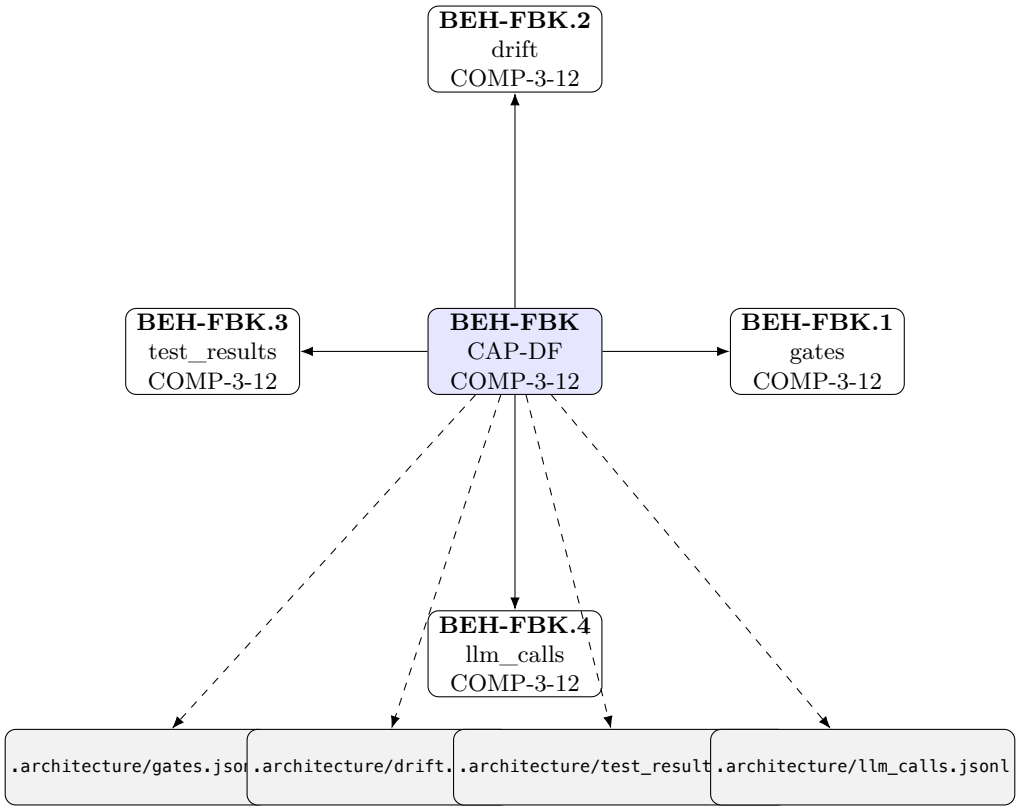
ID	Name	Intent
BEH-SIL.2	append_event	Append a SILRecord event to the SQLite ring buffer, evicting the oldest when the per-component cap is reached.
BEH-SIL.3	compute_rollup	Compute pre-aggregated 7-day metrics (invocations, failure_rate, avg_duration_ms) per component and emit rollup snapshot YAML.
BEH-SIL.4	query_component_health	Answer a single-component health query returning the SILRecord + 7-day trend, backing the OCA architect_component_health MCP tool.
BEH-SIL.5	attach_evidence_to_promise	Attach SIL evidence (recent events, metric summary) to a promise's verified_by chain so runtime behavior is auditable back to t…
BEH-SIL.6	no_op_when_unbound	When no SILStore is bound to the current context, the decorator becomes a no-op so instrumentation adds zero overhead in unboun…
BEH-SIL.7	audit_decorator_coverage	Audit that every architecturally-significant call site is decorated, failing CI if instrumentation drifts.
BEH-SIL.8	async_call_instrumentation	Instrument async call sites correctly so start/end timestamps span the await, not the coroutine construction.

BEH-FBK — append_and_read_feedback_journals

Provide append-only JSONL journals under .architecture/ that capture gate outcomes, drift snapshots, test results, and LLM call provenance so the model has evidence to close its own quality loop.

Realizes: `CAP-DF`

Flow



Promises

- **intent** *(verified)* — Provide append-only JSONL journals under .architecture/ that capture gate outcomes, drift snapshots, test results, and LLM call provenance so the model has evidence to close its own quality loop.
- **goals** *(verified)* — Zero data loss on append (fsync every record); Fail-soft on read (missing files → empty list, malformed lines skipped); Byte-stable digests when AMS_DETERMINISTIC_NOW is set
- **requirements** *(verified)* — Journals live at .architecture/<gates|drift|test_results|llm_calls>.jsonl; Append uses O_APPEND | O_CREAT | O_WRONLY plus fsync; Read returns empty list when file is absent; Malformed lines are silently skipped, not r…

Direct sub-behaviors (4)

ID	Name	Intent
`BEH-FBK.1`	`append\_gate\_event`	Append a GateEvent record to .architecture/gates.jsonl every time the OCA architect_gate MCP tool runs.
`BEH-FBK.2`	`append\_drift\_snapshot`	Append a drift snapshot (broken_ref, unrealized_capability, orphan, missing_impl) to .architecture/drift.jsonl after every pipe…

ID	Name	Intent
BEH-FBK.3	ingest_test_results	Ingest a JUnit XML report as a TestResult batch and append to .architecture/test_results.jsonl, backing the opencode-arch feedb…
BEH-FBK.4	append_llm_call_record	Append an LLMCallRecord to .architecture/llm_calls.jsonl on every provider call so the prompt-refinement walk has evidence for …

Appendix: Relationships

Total: 54 edges

- `contains`: 47
- `realizes`: 7